OPIPE-02: Span Processing & Enrichment

> **Series:** OPIPE — OpenPipeline Beyond Logs | **Notebook:** 2 of 7 | **Created:** March 2026 | **Last Updated:** 10/26/2026

Filtering, Enriching, and Routing Distributed Traces at Ingestion

Span data from distributed tracing can be high-volume and noisy. Health check probes, readiness endpoints, and internal synthetic calls generate spans that consume storage but rarely provide troubleshooting value. OpenPipeline's **Spans scope** lets you filter, enrich, and route trace data before it reaches Grail — reducing cost and improving signal-to-noise ratio.

This notebook covers the span-specific capabilities of OpenPipeline. For general pipeline architecture, see **OPIPE-01: OpenPipeline as a Multi-Scope Platform**. For querying spans after processing, see **SPANS-01: Fundamentals**.

Table of Contents

1. Understanding Span Data in OpenPipeline
2. Span Filtering: Dropping Noise at Ingestion
3. Span Attribute Enrichment
4. Span-to-Event Extraction
5. Routing Spans to Dedicated Buckets
6. Monitoring Your Span Pipeline
7. Summary
8. Next Steps
9. References

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail and distributed tracing enabled
Permissions	`storage:spans:read`; `settings:objects:write` on `builtin:openpipeline.spans.pipelines` (OpenPipeline configuration is stored as Settings objects). Routing changes need `settings:objects:admin` or a policy granting write on `builtin:openpipeline.spans.routing`.
Data	At least 1 hour of span data from instrumented services
Recommended	**OPIPE-01** (multi-scope architecture) and **SPANS-01** (span fundamentals)

1. Understanding Span Data in OpenPipeline

Before configuring span processing, you need to understand what fields are available for routing and filtering decisions.

Key Span Fields for Pipeline Configuration

Field	Description	Pipeline Use
`span.kind`	`server`, `client`, `internal`, `producer`, `consumer`	Route by span type
`service.name`	Logical service name set by the OpenTelemetry SDK — present on OTel spans only	Route OTel spans by service
`dt.service.name`	Dynatrace service name from service detection, *"equal to the Smartscape service node name"* — present on every span	Group and filter by service in DQL
`span.name`	The operation name (e.g., `GET /health`)	Filter noise
`http.route`	The HTTP route pattern	Filter health checks
`http.request.method`	HTTP method (GET, POST, etc.)	Filter by method
`http.response.status_code`	HTTP response code	Filter or extract metrics

Field	Description	Pipeline Use
`db.system`	Database type (mysql, postgresql, redis)	Route DB spans
`k8s.namespace.name`	Kubernetes namespace	Route by environment
`dt.smartscape.service`	Dynatrace service ID (`stable` ; `dt.entity.service` is `deprecated` in the semantic dictionary)	Route by service entity

> **Which service field?** On the validation tenant, `service.name` was set on 21,586 of 322,799 spans in an hour (6.7%, the OTel ones) while `dt.service.name` was set on all of them. A query grouped by `service.name` puts every OneAgent span in one null row, so the DQL cells in this series use `dt.service.name`. For a pipeline matcher, check in the pipeline's sample-data preview that the field you match on is present on the records at that point.
>
> **Dictionary:** `service.name` (`stable`), `dt.service.name` (`stable`), `dt.smartscape.service` (`stable`), `dt.entity.service` (`deprecated`), read 10/02/2026.

Discovering Your Span Landscape

```
// Span volume by kind and service (top 20)
fetch spans, from:-1h
| summarize span_count = count(), by:{span.kind, dt.service.name}
| sort span_count desc
| limit 20
```

```
// Top span operations by volume (candidates for filtering)
fetch spans, from:-1h
| summarize span_count = count(), by:{span.name, dt.service.name}
| sort span_count desc
| limit 20
```

2. Span Filtering: Dropping Noise at Ingestion

Span filtering is usually the highest-impact optimization in the spans scope. In community practice, health checks, probes, scrapes and preflight requests are a large share of span volume in many environments — measure your own share with the query below before writing drop rules.

Common Noise Patterns to Drop

Pattern	Filter Condition	Rationale
Health check probes	`url.path` matches `"*health*"`, `"/ready*"` or `"/livez*"`; `span.name` matches `"grpc.health.v1.Health/*"`	Kubernetes liveness/readiness probes and gRPC health checks
Synthetic monitoring	`span.name` matches `"*synthetic*"`	Internal synthetic test traffic
Internal heartbeats	`span.kind == "internal"` AND `span.name` matches `"*heartbeat*"`	Framework-generated heartbeats
Metrics scraping	`http.route` matches `"/metrics"` or `"/actuator/prometheus"`	Prometheus scrape endpoints
OPTIONS preflight	`http.request.method == "OPTIONS"`	CORS preflight requests

> **Match on the path, not the span name.** HTTP span names often carry only the method (`GET`), with the path in `url.path` / `http.route`, so a `span.name` pattern such as `"GET /health*"` can match nothing. On the validation tenant (10/15/2026, 1 h), 30% of HTTP server spans were named by method alone, and `GET /health*` / `GET /ready*` on `span.name` matched 0 spans while 2,505 health-check spans existed (`/_healthz` paths and `grpc.health.v1.Health/Check`).
>
> **Dictionary:** `url.path` (`stable`), read 10/15/2026.

Measuring the Impact

Before configuring drop rules, quantify how much volume each noise pattern represents:

```
// Quantify noise: health checks, synthetic, and OPTIONS requests
fetch spans, from:-1h
| summarize total = count(),
    health_checks = countIf(matchesValue(url.path, {"*health*", "/ready*",
"/livez*"}) or matchesValue(span.name, "grpc.health.v1.Health/*")),
    metrics_scrapes = countIf(matchesValue(url.path, {"/metrics*", "/actuator/*"})),
    options_preflight = countIf(http.request.method == "OPTIONS")
| fieldsAdd noise_total = health_checks + metrics_scrapes + options_preflight
| fieldsAdd noise_pct = round(toDouble(noise_total) / toDouble(total) * 100,
decimals: 1)
```

```
// Detail: Top 10 span names that look like noise (short-lived, high-frequency)
fetch spans, from:-1h
| summarize {span_count = count(), avg_duration = avg(duration)}, by:{span.name}
| sort span_count desc
| limit 10
| fieldsAdd avg_duration_ms = round(avg_duration / 1ms, decimals: 2)
```

3. Span Attribute Enrichment

Enrichment adds fields to spans at ingestion that would be expensive or impossible to add at query time. Common enrichment patterns:

Business Context Enrichment

Add business-meaningful attributes based on technical fields:

Condition	Enrichment Field	Value	Purpose
`dt.service.name` starts with `"checkout"`	`business.domain`	`"commerce"`	Business domain tagging
`matchesValue(k8s.namespace.name, "*prod*")`	`environment`	`"production"`	Environment classification
`http.response.status_code` >= 500	`incident.severity`	`"high"`	Severity pre-classification
`db.system` is not null	`span.category`	`"database"`	Span categorization

Security Context Enrichment

Add `dt.security_context` based on service ownership (see **OPIPE-01** and **ORGNZ-06** for details):

Condition	`dt.security_context` Value
`k8s.namespace.name` matches `"team-alpha-*"`	`"team-alpha"`
`dt.service.name` matches `"payment-*"`	`"payment-team"`
`db.system` is not null	`"database-team"`

Verifying Enrichment

```
// Check which spans have security context enrichment
fetch spans, from:-1h
| summarize {total = count(), with_security_context =
countIf(isNotNull(dt.security_context))},
    by:{dt.service.name}
| fieldsAdd coverage_pct = round(toDouble(with_security_context) / toDouble(total) *
100, decimals: 1)
| sort total desc
| limit 15
```

4. Span-to-Event Extraction

OpenPipeline can extract **business events** and **SDLC events** (Data extraction stage) or **Davis events** (Davis stage) from span data at ingestion. It does not generate log records from spans. This is useful for:

- **Error alerting**: Extract a Davis event when a span has `http.response.status_code >= 500`, sending it to root cause analysis without querying spans directly
- **Audit trail**: Extract a business event for every span that touches a sensitive service
- **SLO tracking**: Extract business events from spans that represent key user transactions

> **Sources:** Data extraction stage (DT docs) — *"In the Data extraction stage, you can extract new records from incoming records that match a condition and re-ingest them as a different data type into another pipeline."*; Davis stage (DT docs) — *"In the Davis stage, you can extract Davis events from incoming records that match a condition and send them to root cause analysis."*

Generation vs. Metric Extraction

Feature	Span-to-Event Extraction	Metric Extraction from Spans
Output	Individual event records	Aggregated metric data points
Use case	Alerting, audit trails	Dashboards, SLOs, trend analysis
Volume	1 event per matching span	1 data point per aggregation interval
Cost	Higher (more records)	Lower (aggregated)

Metric extraction from spans is covered in detail in **OPIPE-03: Sampling-Aware Metrics**.

```
// Identify error spans that could trigger event generation
fetch spans, from:-1h
| filter http.response.status_code &gt;= 500
| summarize error_count = count(), by:{dt.service.name, http.response.status_code}
| sort error_count desc
| limit 15
```

5. Routing Spans to Dedicated Buckets

Just as logs benefit from bucket separation (see **OPLOGS-04: Buckets & Data Governance**), spans benefit from targeted routing.

Span Routing Strategy

Pipeline	Routing Condition	Target Bucket	Retention	Rationale
`frontend-traces`	`span.kind == "server"` AND `service.name` matches frontend	`frontend_spans`	14 days	High volume, short-term debugging
`api-traces`	`span.kind == "server"` AND `http.route` starts with `"/api"`	`api_spans`	35 days	API SLO tracking
`database-traces`	`span.kind == "client"` AND `db.system` is not null	`db_spans`	14 days	Query performance analysis
`messaging-traces`	`span.kind == "producer" or span.kind == "consumer"` (`in()` is not enabled in OpenPipeline matchers)	`messaging_spans`	14 days	Async flow debugging

Pipeline	Routing Condition	Target Bucket	Retention	Rationale
Default route	Everything else	`default_spans`	Built-in (10 days on the validation tenant — read yours with `fetch dt.system.buckets`)	Catch-all

Routes are evaluated in order and the first match wins, so a frontend service's `/api` server span goes to `frontend-traces` here; put the more specific route first if that is not what you want. The route chooses the pipeline; the pipeline's **Bucket assignment** stage chooses the bucket.

Verifying Bucket Distribution

```
// Current span bucket distribution
fetch spans, from:-1h
| summarize span_count = count(), by:{dt.system.bucket}
| sort span_count desc
```

```
// Span volume by kind and bucket (is routing working correctly?)
fetch spans, from:-1h
| summarize span_count = count(), by:{span.kind, dt.system.bucket}
| sort span_count desc
```

6. Monitoring Your Span Pipeline

After configuring span processing rules, monitor their effectiveness over time.

```
// Span volume trend by pipeline (are drop rules reducing volume?)
fetch spans, from:-24h
| makeTimeseries span_count = count(), by:{dt.openpipeline.pipelines}, interval:1h
```

```
// Span volume by service over 24h (spot unexpected spikes)
fetch spans, from:-24h
| makeTimeseries span_count = count(), by:{dt.service.name}, interval:1h
```

Seeing What Was Dropped: Self-Monitoring Metrics

Counting stored spans by `dt.openpipeline.pipelines` cannot, by construction, see records that a drop rule removed or that were never stored — which is exactly what a drop rule produces. OpenPipeline publishes **self-monitoring metrics** for that:

`dt.sfm.openpipeline.pipelines_out.records` counts records leaving each pipeline, split by `pipeline_id` and `bucket_name`, and `dt.sfm.openpipeline.not_stored.records` counts records that were not stored, split by `configuration` (scope), `pipeline_id`, and `reason`.

> **Sources:** OpenPipeline self-monitoring metrics (DT docs).

```
// Records leaving each span pipeline, by destination bucket (self-monitoring)
timeseries records = sum(dt.sfm.openpipeline.pipelines_out.records), from:-24h,
  filter:{configuration == "spans"}, by:{pipeline_id, bucket_name}
| fieldsAdd total = arraySum(records)
| fields pipeline_id, bucket_name, total
| sort total desc
```

```
// Records not stored, by scope and reason (drops, no-storage assignments, invalid
records)
timeseries records = sum(dt.sfm.openpipeline.not_stored.records), from:-24h,
  by:{configuration, reason}
| fieldsAdd total = arraySum(records)
| fields configuration, reason, total
| sort total desc
```

Summary

In this notebook you learned:

- **Span landscape discovery** — Key fields for routing and filtering decisions
- **Noise filtering** — Dropping health checks, synthetic traffic, and preflight requests at ingestion
- **Attribute enrichment** — Adding business context and security context to spans
- **Event extraction** — Extracting business, SDLC, and Davis events from span data for alerting and audit
- **Bucket routing** — Separating spans by purpose with differentiated retention
- **Pipeline monitoring** — Tracking volume trends, and using OpenPipeline self-monitoring metrics to see records that were dropped or not stored

---

Next Steps

Continue to **OPIPE-03: Sampling-Aware Metrics** to learn how to extract accurate metrics from sampled span data — including the RED metrics (Rate, Errors, Duration) that drive SLOs and dashboards.

References

- Extraction stages in OpenPipeline (DT docs)
- Processing in OpenPipeline (DT docs)
- Data extraction stage (DT docs)
- Davis stage (DT docs)
- OpenPipeline self-monitoring metrics (DT docs)
- Organize your data stored in Grail (DT docs)
- Migrate OpenPipeline configurations to Settings API (DT docs) — *"Routing management is restricted to administrators (* `settings:objects:admin` *)."*

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.